COSC2637/2633 Big Data Processing
Lab 2: Tutorial/Lab class (week 3)

Objectives

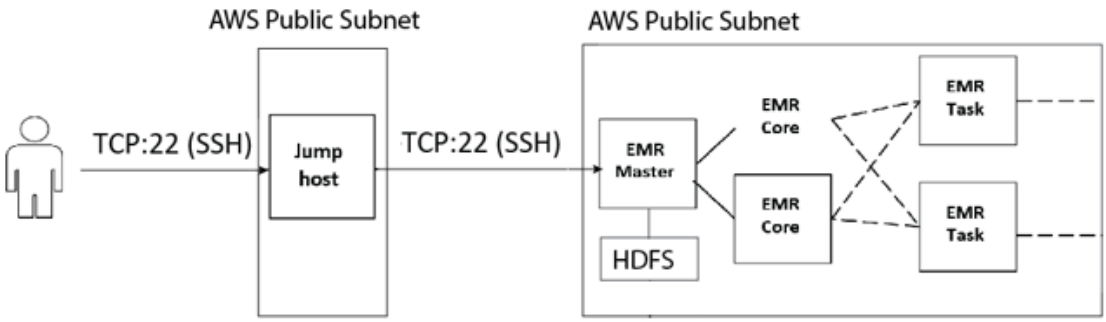
- Create an AWS EMR cluster and understand the basics of the cluster
- Build a MapReduce program using Python
- Upload MapReduce program to AWS EMR cluster
- Run MapReduce program on AWS EMR cluster

● Create Cluster and Cluster Basics

Follow same procedure as given in week 2 tutorial/lab document. The Amazon EMR Management Guide gives a brief explanation of clusters and nodes.
https://docs.aws.amazon.com/cli/latest/reference/emr/

The central component of Amazon EMR is the cluster. A cluster is a collection of Amazon Elastic Compute Cloud (Amazon EC2) instances. Each instance in the cluster is called a node. Each node has a role within the cluster, referred to as the node type. Amazon EMR also installs different software components on each node type, giving each node a role in a distributed application like Apache Hadoop. The node types in Amazon EMR are as follows:

- **Master node:** A node that manages the cluster by running software components to coordinate the distribution of data and tasks among other nodes for processing. The master node tracks the status of tasks and monitors the health of the cluster. Every cluster has a master node, and it's possible to create a single-node cluster with only the master node.
- **Core node:** A node with software components that run tasks and store data in the Hadoop Distributed File System (HDFS) on your cluster. Multi-node clusters have at least one core node.
- **Task node:** A node with software components that only runs tasks and does not store data in HDFS. Task nodes are optional.



To obtain the basic information of your cluster, first find your cluster id:

"Id": "j-xxxxxxxxxxxx"

by running the following in command line

aws emr list-clusters --active --region us-east-1

```
[ec2-user@ip-192-168-10-47 ~]$ aws emr list-clusters --active --region us-east-1
```

```
…
"Status": {
          "Timeline": {
                    "ReadyDateTime": 1564630716.927,
                    "CreationDateTime": 1564629351.897
           },
           "State": "WAITING",
           "StateChangeReason": {
                    "Message": "Cluster ready to run steps."
           }
},
"NormalizedInstanceHours": 42,
"Id": "j-xxxxxxxxxxxx",
"Name": "sxxxxxx"
….
```

(Note “Name”: “sxxxxxxx” above where “sxxxxxxx” is your account)

aws emr describe-cluster --cluster-id j-xxxxxxxxx --region us-east-1

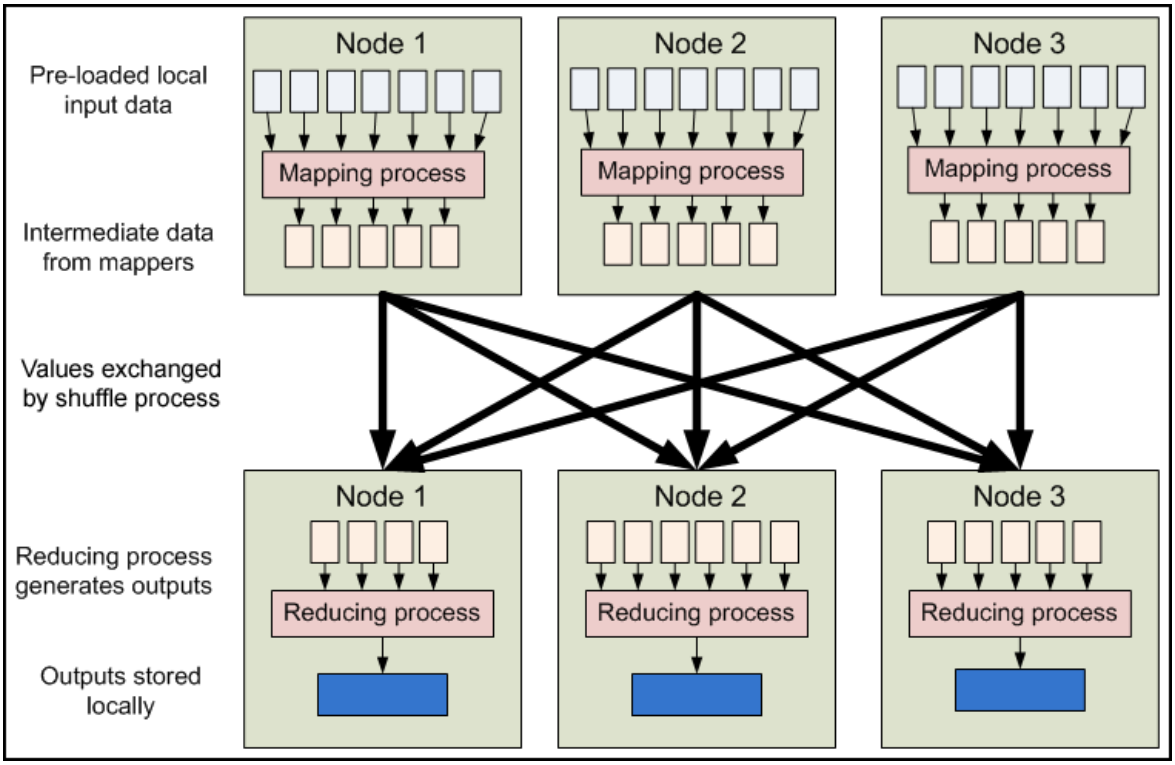
```
[ec2-user@ip-192-168-10-47 ~]$ aws emr describe-cluster --cluster-id j-IYZBXLRRJY25 --region us-east-1
```

```
"Name": "Master",
"InstanceGroupType": "MASTER",
"EbsBlockDevices": [
    {
        "Device": "/dev/sdb",
        "VolumeSpecification": {
            "VolumeType": "gp2",
            "SizeInGB": 32
        }
    }
],
"ShrinkPolicy": {},
"Id": "ig-121ZK6BZOT6ZB",
"LastSuccessfullyAppliedConfigurations": [],
"LastSuccessfullyAppliedConfigurationsVersion": 0,
"ConfigurationsVersion": 0,
"Configurations": [],
"InstanceType": "m4.large",
```

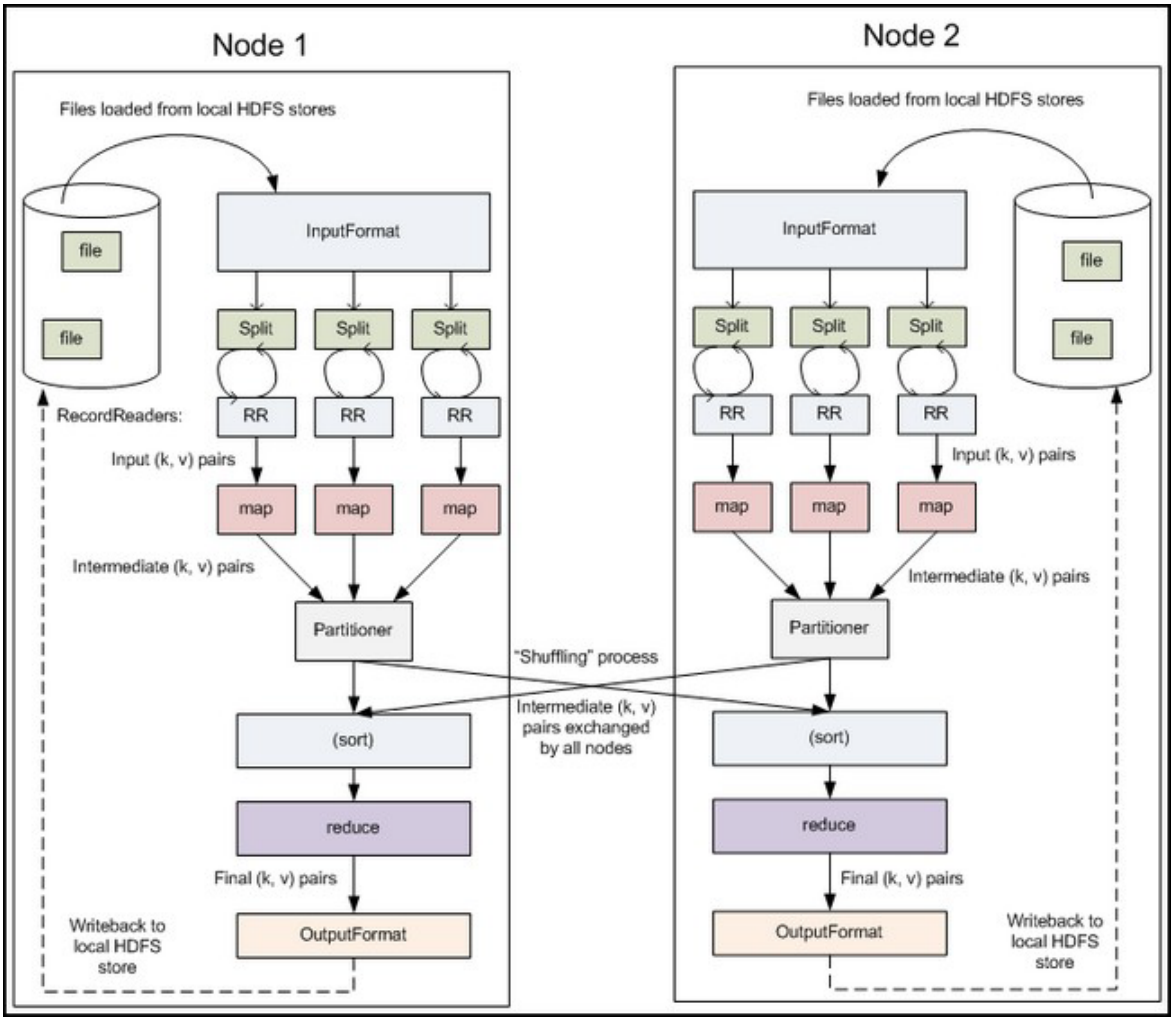
- **Map-Reduce high level workflow**

Diagrams from the following tutorial:

https://developer.yahoo.com/hadoop/tutorial/module4.html#dataflow

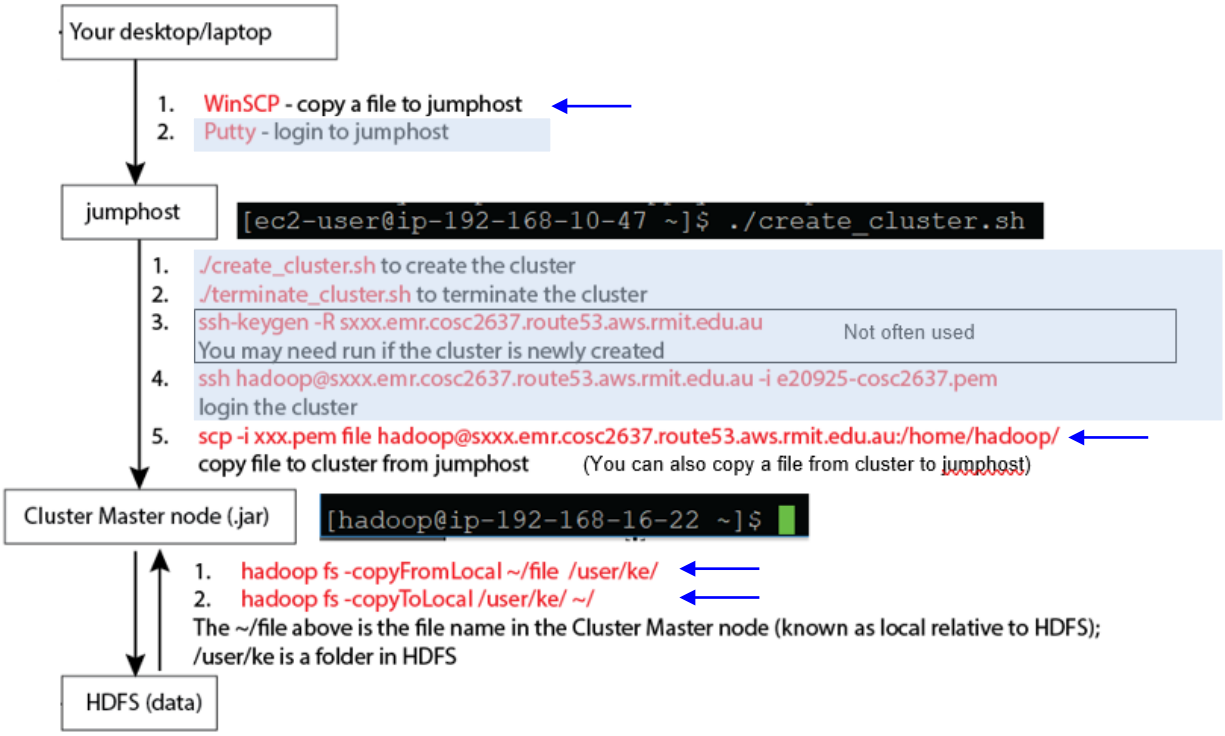


1) Input Format
2) Input Splits
3) Record Reader
4) Mapper
5) Partition and Shuffle
    - Partition – subset based on keys
    - Shuffling – moving map outputs to the reducers
6) Sort
7) Reduce
8) Output Format

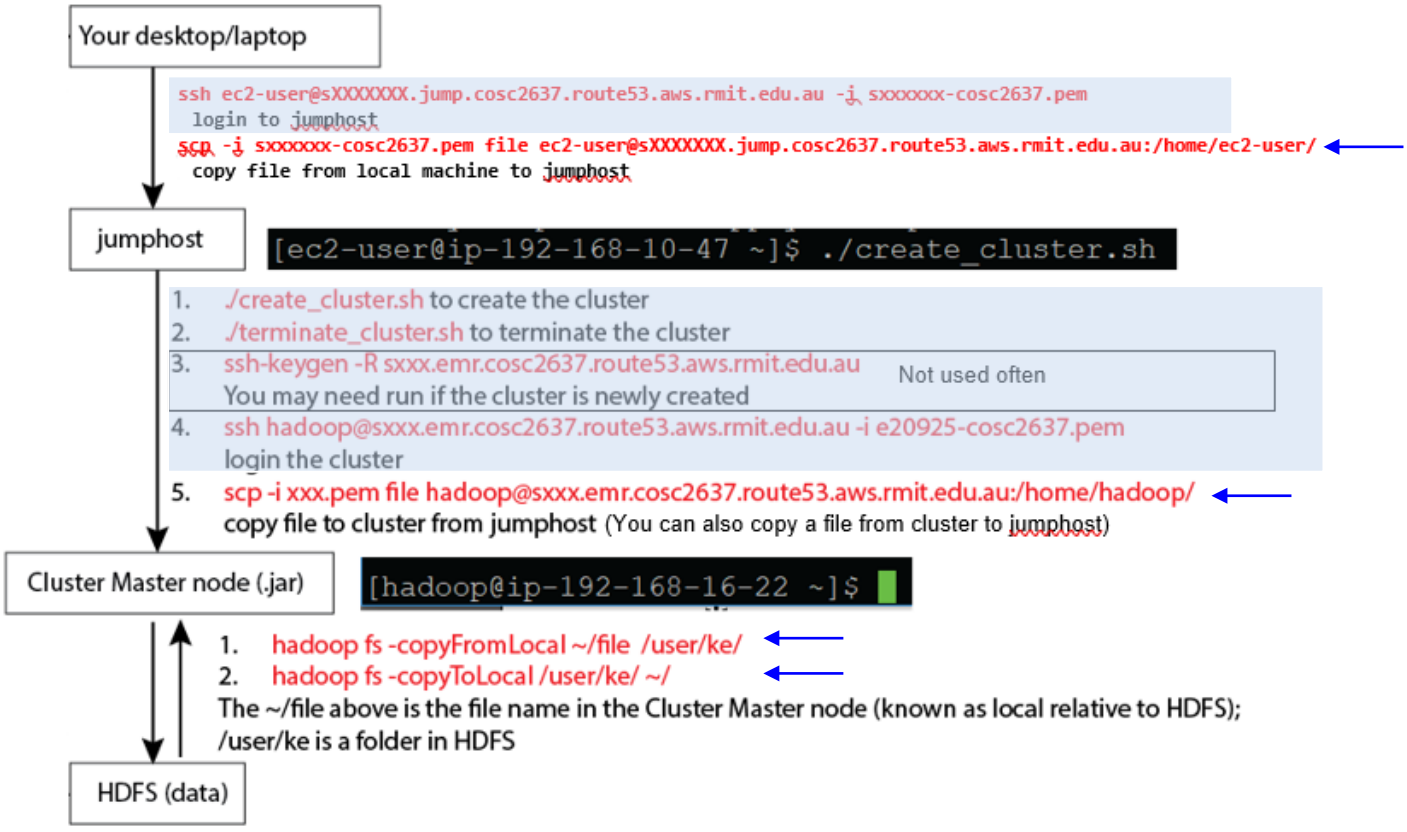


- **Copy files between your laptop and HDFS via Jumphost**

  - Windows user



  - Mac/Linux/Unix user



The following steps can be used to copy files from/to HDFS via jumphost:

1. Copy files between your laptop and jumphost.
   - For Windows users, use WinSCP in the same way as copying the key file sxxxxxx-cosc2637.pem from your laptop to jumphost. It is the same to copy files from jumphost to your laptop by drag-and-drop in the opposite direction.
   - For Mac/Linux/Unix users, use the scp command in the same way as copying the key file sxxxxxx-cosc2637.pem from your laptop to jumphost:

     $ scp -i sxxxxx-cosc2637.pem source destination

     Changing the order of source and destination in the scp command will copy files from jumphost to your laptop.
2. Suppose the files are already in jumphost. You can use the scp command in jumphost terminal to copy the files from jumphost (e.g., all files in folder ./data/) to the cluster Master node (e.g., to folder /home/hadoop).

$ scp -r -i sxxxxx-cosc2637.pem ./data/* hadoop@sxxxxx.emr.cosc2637.route53.aws.rmit.edu.au:/home/hadoop/

An example (note the key sxxxxx-cosc2637.pem must be in access mode 400 or 600):

```
[ec2-user@ip-192-168-4-64 ~]$ scp -r -i e20925a-cosc2637.pem ./data/* hadoop@e20
925a.emr.cosc2637.route53.aws.rmit.edu.au:/home/hadoop/
The authenticity of host 'e20925a.emr.cosc2637.route53.aws.rmit.edu.au (192.168.
29.91)' can't be established.
ECDSA key fingerprint is SHA256:W7m1a96Hy68FaWSLG++eGp4r2pMmzDsF6tT/FW60R9w.
ECDSA key fingerprint is MD5:f6:31:a3:71:67:03:70:0f:07:d5:b2:b8:1a:b1:e6:a4.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'e20925a.emr.cosc2637.route53.aws.rmit.edu.au,192.168
.29.91' (ECDSA) to the list of known hosts.
hadoop-streaming-3.1.4.jar                    100%  172KB   3.7MB/s   00:00
task3-mapperInMapCombiner.py                  100%  635   209.3KB/s   00:00
task3-reducer.py                              100% 1481   633.0KB/s   00:00
task3-run.sh                                  100%  390   164.6KB/s   00:00
trip.txt                                      100%   85KB  17.8MB/s   00:00
```

Suppose you have files on cluster Master node and you want to copy them to jumphost. You can use the scp command in jumphost terminal by changing the order of the source and destination. Let us copy the files from the cluster Master node (e.g., all files in folder /home/hadoop/) to jumphost (e.g., folder ./data/).

$ scp -r -i sxxxxx-cosc2637.pem hadoop@sxxxxx.emr.cosc2637.route53.aws.rmit.edu.au:/home/hadoop/* ./data/

An example (note the key sxxxxx.pem must be in access mode 400):

```
[ec2-user@ip-192-168-4-64 ~]$ scp -r -i e20925a-cosc2637.pem hadoop@e20925a.emr.
cosc2637.route53.aws.rmit.edu.au:/home/hadoop/* ./data/
hadoop-streaming-3.1.4.jar                    100%  172KB  64.2MB/s   00:00
task3-mapperInMapCombiner.py                  100%  635    610.3KB/s   00:00
task3-reducer.py                              100% 1481     1.8MB/s   00:00
task3-run.sh                                  100%  390    436.2KB/s   00:00
trip.txt                                      100%   85KB  52.8MB/s   00:00
```

3. Copy the files from the cluster Master node (e.g., ./trip.txt but you can specify a list of file names here) to HDFS (e.g., folder /). You can use

$ hadoop fs -copyFromLocal ./trip.txt / or

$ hadoop fs -put ./trip.txt /

An example:

```
[hadoop@ip-192-168-29-91 ~]$ hadoop fs -copyFromLocal ./trip.txt /
[hadoop@ip-192-168-29-91 ~]$ hadoop fs -ls /
Found 5 items
drwxr-xr-x   - hdfs   hdfsadmingroup          0 2023-07-30 03:40 /apps
drwxrwxrwt   - hdfs   hdfsadmingroup          0 2023-07-30 03:43 /tmp
-rw-r--r--   1 hadoop hdfsadmingroup      86992 2023-07-30 04:26 /trip.txt
drwxr-xr-x   - hdfs   hdfsadmingroup          0 2023-07-30 03:40 /user
drwxr-xr-x   - hdfs   hdfsadmingroup          0 2023-07-30 03:40 /var
```

Copy the files from HDFS to the cluster Master node. You can use:

$ hadoop fs -copyToLocal /trip.txt /trip1.txt ./ or

$ hadoop fs -get /trip.txt /trip1.txt ./

An example:

```
[hadoop@ip-192-168-29-91 ~]$ hadoop fs -copyToLocal /trip.txt /trip1.txt ./
[hadoop@ip-192-168-29-91 ~]$ ls
hadoop-streaming-3.1.4.jar    task3-reducer.py  trip1.txt
task3-mapperInMapCombiner.py  task3-run.sh      trip.txt
```

$ hadoop fs -getmerge /trip.txt /trip1.txt ./trip2.txt (merge files into one file)

An example:

```
[hadoop@ip-192-168-29-91 ~]$ hadoop fs -get /trip.txt ./trip.txt
[hadoop@ip-192-168-29-91 ~]$ hadoop fs -get /trip1.txt ./trip1.txt
[hadoop@ip-192-168-29-91 ~]$ hadoop fs -getmerge /trip.txt /trip1.txt ./trip2.txt
[hadoop@ip-192-168-29-91 ~]$ wc -l trip.txt
9999 trip.txt
[hadoop@ip-192-168-29-91 ~]$ wc -l trip1.txt
9999 trip1.txt
[hadoop@ip-192-168-29-91 ~]$ wc -l trip2.txt
19998 trip2.txt
```

- **Manage Files in HDFS from Cluster Master Node using "hadoop fs"**

COMMAND	DESCRIPTION
-ls	List files with permissions and other details
-mkdir	Creates a directory named path in HDFS
-rm	To Remove File or a Directory
-rmdir	Delete a directory
-put	Upload a file / Folder from the local disk to HDFS
-cat	Display the contents for a file
-du	Shows the size of the file on hdfs.
-get	Store file / Folder from HDFS to local file
-getmerge	Merge Multiple Files in an HDFS and store it from HDFS to local file
-count	Count number of directory, number of files and file size
-setrep	Changes the replication factor of a file – "hadoop fs -setrep 3 /trip.txt"
-stat %r	Check the replication factor of a file – "hadoop fs -stat %r /trip.txt"
-mv	HDFS Command to move files from source to destination
-moveFromLocal	Move file / Folder from local disk to HDFS
-moveToLocal	Move a File to HDFS from Local
-cp	Copy files from source to destination
-tail	Displays last kilobyte of the file
-touchz	Create a new file on HDFS with size 0 bytes
-appendToFile	Appends the content to the file which is present on HDF
-copyFromLocal	Copy file from local file system
-copyToLocal	Copy files from HDFS to local file system
-usage	Return the Help for Individual Command – "hadoop fs -usage ls"
-checksum	Returns the checksum information of a file – "hadoop fs -checksum /trip.txt"
-chgrp	Change group association of files/change the group of a file or a path
-chmod	Change the permissions of a file
-chown	change the owner and group of a file
-df	Displays free space
-head	Displays the first kilobyte of the file
-stat	Print statistics about the file/directory
-truncate	Truncate all files that match the specified file pattern to the specified length
-find	find the files in the HDFS file system – "hadoop fs -find / hadoop"

- **First Python MapReduce Program**

1. Suppose you have created the AWS EMR cluster via ./create_cluster.sh in jumphost
2. Develop mapper.py and reducer.py using any text editor you prefer like (source code provided to save your time).

mapper.py

```
#!/usr/bin/env python3
import sys
# Word Count Example
# input comes from standard input STDIN
for line in sys.stdin:
    line = line.strip() #remove leading and trailing whitespaces
    words = line.split(" ") #split the line into words and returns as a list

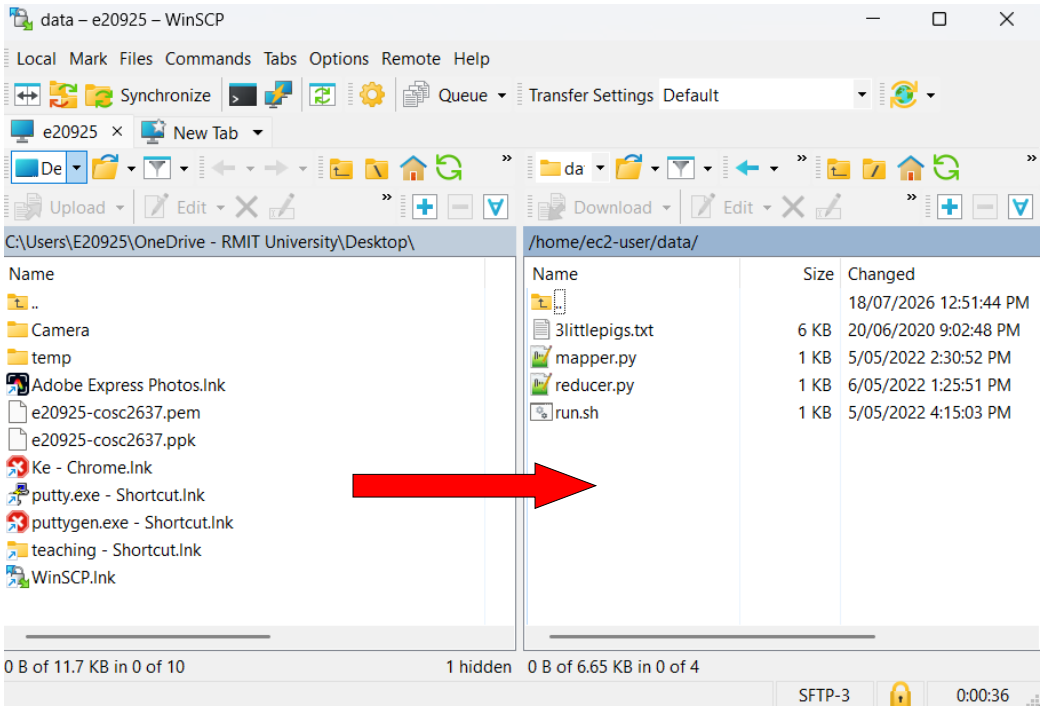
    for word in words:
        word = ''.join(c for c in word if c.isalnum()) # output alphanumeric item only
        if word.strip():
            print('%s\t%s' % (word,"1")) #Emit <key, value> pairs
```

reducer.py

```
#!/usr/bin/env python3
import sys
#from operator import itemgetter
# using a dictionary to map words to their counts
current_word = None
current_count = 0
word = None
# input comes from STDIN
for line in sys.stdin:
    line = line.strip()
    word,count = line.split('\t')
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
        current_count = count
        current_word = word
if current_word == word:
    print('%s\t%s' % (current_word,current_count))
```

3. Upload to jumphost the following files

- mapper.py
- reducer.py
- run.sh *(script running the MapReduce program)*
- 3littlepigs.txt *(data to be processed)*



Then, copy them to the cluster master node using scp.

For example, the files are stored in folder ./data/ in jumphost, they will be transmitted to master node in folder /home/hadoop/.

```
[ec2-user@ip-192-168-13-73 ~]$ scp -r -i e20925-cosc2637.pem ./data/* hadoop@e20
925.emr.cosc2637.route53.aws.rmit.edu.au:/home/hadoop/
3littlepigs.txt                               100% 5485     3.4MB/s   00:00
mapper.py                                     100%  470   323.6KB/s   00:00
reducer.py                                    100%  640   517.2KB/s   00:00
run.sh                                        100%  222   164.6KB/s   00:00
[ec2-user@ip-192-168-13-73 ~]$
```

#scp -r -i e20925-cosc2637.pem data/* hadoop@e20925.emr.cosc2637.route53.aws.rmit.edu.au:/home/hadoop/

4. Now login Master node from jumphost.



#ssh hadoop@e20925.emr.cosc2637.route53.aws.rmit.edu.au -i e20925-cosc2637.pem

5. Make sure the data file (3littlepigs.txt) to be processed in HDFS

   # hadoop fs -copyFromLocal 3litterpigs.txt /

   To check the file is already in the specified folder ("/" in this case) on HDFS.

   # hadoop fs -ls /

```
[hadoop@ip-192-168-25-224 ~]$ ls
3littlepigs.txt  mapper.py  reducer.py  run.sh
[hadoop@ip-192-168-25-224 ~]$ hadoop fs -put 3littlepigs.txt /
[hadoop@ip-192-168-25-224 ~]$ hadoop fs -ls /
Found 5 items
-rw-r--r--   1 hadoop hdfsadmingroup       5485 2026-07-18 03:13 /3littlepigs.tx
t
drwxr-xr-x   - hdfs   hdfsadmingroup          0 2026-07-18 00:22 /apps
drwxrwxrwt   - hdfs   hdfsadmingroup          0 2026-07-18 00:27 /tmp
drwxr-xr-x   - hdfs   hdfsadmingroup          0 2026-07-18 00:22 /user
drwxr-xr-x   - hdfs   hdfsadmingroup          0 2026-07-18 00:22 /var
[hadoop@ip-192-168-25-224 ~]$
```

6. "run.sh" is script to run the MapReduce program. Note the second line specifies a jar file which is provided by the cluster.

new 2 | run.sh

```
#!/bin/bash
hadoop jar /usr/hadoop/lib/hadoop-streaming.jar \
-D mapred.reduce.tasks=3 \
-file ./mapper.py \
-mapper ./mapper.py \
-file ./reducer.py \
-reducer ./reducer.py \
-input /3littlepigs.txt \
-output /output
```

7. copy it master node as above. Make sure run.sh is runnable using "chmod +x run.sh"

```
[hadoop@ip-192-168-25-224 ~]$ ll
total 20
-rw-rw-r--. 1 hadoop hadoop 5485 Jul 18 03:08 3littlepigs.txt
-rw-rw-r--. 1 hadoop hadoop  470 Jul 18 03:08 mapper.py
-rw-rw-r--. 1 hadoop hadoop  640 Jul 18 03:08 reducer.py
-rw-rw-r--. 1 hadoop hadoop  222 Jul 18 03:08 run.sh
[hadoop@ip-192-168-25-224 ~]$ chmod +x run.sh
[hadoop@ip-192-168-25-224 ~]$ ll
total 20
-rw-rw-r--. 1 hadoop hadoop 5485 Jul 18 03:08 3littlepigs.txt
-rw-rw-r--. 1 hadoop hadoop  470 Jul 18 03:08 mapper.py
-rw-rw-r--. 1 hadoop hadoop  640 Jul 18 03:08 reducer.py
-rwxrwxr-x. 1 hadoop hadoop  222 Jul 18 03:08 run.sh
[hadoop@ip-192-168-25-224 ~]$
```

8. By executing "run.sh", you can run your first MapReduce program using "hadoop-streaming-3.1.4.jar" to process txt file "3littlepigs.txt" on HDFS.

```
[hadoop@ip-192-168-25-224 ~]$ ./run.sh
```

Execution is successful if you can see something like

```
22/05/05 05:50:20 INFO mapreduce.Job:  map 0% reduce 0%
22/05/05 05:50:31 INFO mapreduce.Job:  map 25% reduce 0%
22/05/05 05:50:41 INFO mapreduce.Job:  map 50% reduce 0%
22/05/05 05:50:44 INFO mapreduce.Job:  map 100% reduce 0%
22/05/05 05:50:49 INFO mapreduce.Job:  map 100% reduce 33%
22/05/05 05:50:55 INFO mapreduce.Job:  map 100% reduce 100%
22/05/05 05:50:56 INFO mapreduce.Job: Job job_1651707531556_0001 completed successfu
lly
22/05/05 05:50:56 INFO mapreduce.Job: Counters: 51
        File System Counters
                FILE: Number of bytes read=3854
                FILE: Number of bytes written=2477899
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=25580
                HDFS: Number of bytes written=3379
                HDFS: Number of read operations=33
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=6
```

9. Check the output using

hadoop fs -cat /output/part-00000

hadoop fs -cat /output/part-00001

hadoop fs -cat /output/part-00002

```
[hadoop@ip-192-168-19-103 ~]$ hadoop fs -cat /output/part-00000
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/lib/hadoop/lib/slf4j-log4j12-1.7.25.jar!/org/
slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/share/aws/emr/emrfs/lib/slf4j-log4j12-1.7.12.
jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
All     1
Along   1
And     4
Autumn  1
But     4
Clunk   3
Excited 1
From    2
He      2
Id      1
```

Note in run.sh, we specify 3 reducer tasks. For each task, the output file is named “part-0000x” where x = 1, 2, 3. You can view them by

Hadoop fs -ls /output/

- **More Features of Python MapReduce Program**
  1. You may want to pass argument(s) to mapper.py. For example, only output words whose length is greater than a threshold without modifying mapper.py.
  2. You want to automatically delete output and input folder on HDFS.

new 2 | run.sh | runArgument.sh

```
#!/bin/bash
arg=4

hadoop fs -rm -f -r /input
hadoop fs -rm -f -r /output
hadoop fs -mkdir /input
hadoop fs -copyFromLocal ./3littlepigs.txt /input/3littlepigs.txt

hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
-D mapred.reduce.tasks=3 \
-file ./mapperArgument.py \
-mapper "./mapperArgument.py $arg" \
-file ./reducer.py \
-reducer ./reducer.py \
-input /input/3littlepigs.txt \
-output /output
```

define an argument

Delete folders on HDFS

Create a new folder on HDFS and copy a file to the folder

define an argument

The mapper.py needs to be updated as well.

```
#!/usr/bin/env python3
import sys
# Word Count Example
# input comes from standard input STDIN

length = int(sys.argv[1])

for line in sys.stdin:
    line = line.strip() #remove leading and trailing whitespaces
    words = line.split(" ") #split the line into words and returns as a list

    for word in words:
        word = ''.join(c for c in word if c.isalnum()) # output alphanumeric item
        only

        if len(word) < length:
            #print('%d\t%s' % (len(word), word))
            continue

        if word.strip():
            print('%s\t%s' % (word,"1")) #Emit <key, value> pairs
```

Read the argument

Use the argument as threshold. If a word's length is less than the threshold, it is omitted.

Don't forget to terminate cluster

$./terminate_cluster.sh